

SHL Assessment Recommendation Agent — Approach Document

Submission for: SHL AI Intern Assignment **Author:** Praasuk Jain **Stack:** Python · FastAPI · Groq Llama 3 (Llama 3.1 8B / 3.3 70B) · Local SentenceTransformers · FAISS · Render

1. Design Choices

Problem framing. Hiring managers start vague ("I need a Java developer") and refine through dialogue. Classical keyword search fails here because it requires the user to already know SHL's vocabulary. I framed this as a retrieval-augmented dialogue system that extracts intent progressively and grounds every recommendation in the catalog.

Stateless architecture. Each `POST /chat` call receives the full conversation history and re-derives all context on the fly. This simplifies deployment (no session store, no race conditions) and matches the assignment spec exactly.

Single LLM call per turn. Rather than a separate intent-classification pass followed by a generation pass, I combined both into one Groq Llama 3 call. The system prompt instructs the model to decide whether to clarify, recommend, refine, or refuse—and output structured JSON directly. This keeps latency well under the 30-second budget (~1–2 s per turn in practice).

Strict catalog grounding. Every name and URL the LLM outputs is validated post-generation against the FAISS-indexed catalog using fuzzy name matching. Invalid recommendations are silently dropped before the response is returned. This makes hallucination impossible to slip through to the evaluator.

2. Retrieval Setup

Embeddings. All 377 assessments are embedded using the local offline **SentenceTransformers (all-MiniLM-L6-v2)** model. Each assessment's searchable text combines its name, description, job levels, languages, test type keys, duration, and adaptive flag into a single document string. Embeddings are cached to disk (`.embeddings_cache.npy`) and baked directly into the Docker image so cold starts are instant and run 100% offline with zero external network API dependencies.

FAISS index. A `IndexFlatIP` (inner-product) index over L2-normalized vectors gives exact cosine-similarity search. At 377 vectors of 384 dimensions, the index fits in ~1.5 MB of RAM and returns results in <0.5 ms.

Hybrid retrieval pipeline:

1. Semantic search over FAISS — retrieves candidates by cosine similarity
2. Metadata filter — narrows by job level, language, and test type if the user specified constraints
3. LLM selection — up to 15 prioritized results are injected into the prompt; the LLM picks

the final 1–10

Two anchor assessments (OPQ32r and Verify G+) are always appended to the retrieved context because they appear in nearly every gold-standard conversation. This ensures the LLM can include them even when the query doesn't semantically retrieve them.

3. Prompt Design

System prompt structure:

- Scope rules (what the agent can and cannot discuss)
- Clarify vs. recommend decision logic (with an explicit instruction not to over-clarify)
- OPQ32r as default personality baseline
- Refinement rules (update the existing list, not restart)
- Turn-budget awareness (force a recommendation if approaching turn 7)
- Output format: strict JSON matching the `ChatResponse` schema
- 7 few-shot examples covering: vague query, specific query, refinement, confirmation, off-topic refusal, legal refusal, and catalog-gap acknowledgment

JSON mode. Groq's JSON object format ensures the model always outputs parseable JSON, eliminating markdown wrapper issues. Temperature is set to 0.2 for factual, consistent recommendations.

Previous recommendations injection. The last assistant turn is parsed to extract existing recommendations, which are formatted and injected into the prompt. This allows the LLM to perform surgical refinements ("add AWS, drop REST") rather than starting over.

4. Evaluation Approach & Iterations

Baseline. Initial testing with a minimal prompt and no catalog injection produced hallucinated assessment names and invented URLs — a hard-eval failure. Adding catalog injection into the prompt context eliminated hallucination.

Recall@10 iteration. Early versions asked too many clarifying questions, burning the 8-turn budget before recommending. I added an explicit instruction: "By turn 3, you must be recommending even if some context is missing." This improved mean Recall@10 from ~0.45 to ~0.72 on the 10 public traces.

What didn't work:

- Separate intent-classification call: doubled latency, hit the 30s timeout on slower turns
- Injecting all 377 assessments: exceeded context limits and confused the model; top-25 retrieval was much better
- Empty recommendations array instead of `null`: broke the evaluator's schema check (fixed)

by explicitly using `null`)

AI tools used: Claude Sonnet (agentic coding for boilerplate generation and debugging); Groq Llama 3 (LLM backbone); SentenceTransformers (local embeddings); manual review of all 10 sample conversations to derive ground-truth and calibrate prompts.

Total implementation: ~15 hours including testing and iteration.